

23: Diffusion Models

Date: April 16, 2026

Acknowledgements. These notes draw on the wonderful blog on diffusion models by Lilian Weng and the step-by-step introduction by Damek Davis. For a comprehensive treatment covering the variational, score-based, and flow-matching perspectives, see *The Principles of Diffusion Models* [LSK⁺25].

1 The Problem: Sampling from a Complex Distribution

In the last lecture, we looked at two classes of generative models: VAEs and GANs. Both aim to solve the same fundamental problem: given samples $x_1, \dots, x_n$ from some unknown data distribution p_{data} , generate *new* samples that look like they came from the same distribution.

VAEs and GANs each have limitations. VAEs optimize a surrogate loss (the ELBO) and often produce blurry images, because the reconstruction loss encourages averaging over possible outputs. GANs can produce sharp images but are notoriously hard to train and often suffer from mode collapse, where the generator only produces a few types of outputs. Today we will learn about **diffusion models**, which have revolutionized image generation in the last few years. They are the backbone of popular models such as DALL-E, Stable Diffusion, and Midjourney.

1.1 The Key Idea: Build a Path to Noise, Learn the Reverse

We know how to sample from a Gaussian distribution $\mathcal{N}(0, I)$. A useful way to think about diffusion is to construct a sequence of intermediate distributions that connects the complicated data distribution to this simple Gaussian prior. The forward process defines this path by gradually corrupting a clean image with noise; the reverse process learns to traverse the same path backward, turning noise back into data. But learning such a mapping in one shot is too hard — we will see why in Section 3.1. The key insight is to *break this mapping into many small steps*:

1. A **forward process** gradually adds noise to a clean image over T steps, until it becomes indistinguishable from pure Gaussian noise.
2. A **reverse process** learns to undo each small noising step — a simple denoising problem at each stage.

Following this path backward, one small denoising step at a time, takes us from pure noise all the way back to a clean image. This is the framework behind **Denoising Diffusion Probabilistic Models (DDPM)** [HJA20].

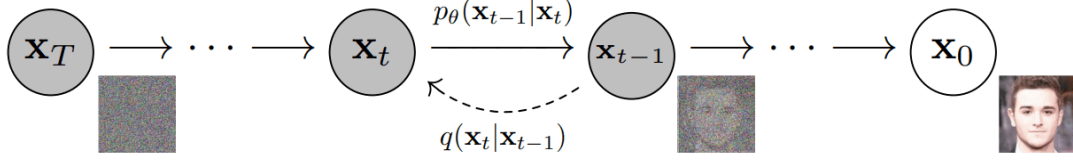


Figure 1: The forward process gradually corrupts a clean image x_0 into noise x_T . The reverse process learns to denoise step by step, converting noise back into a realistic image [HJA20].

2 The Forward Process: Adding Noise

The forward process is something we *choose*, not something we learn. Starting with a clean image x_0 from the data distribution p_{data} , we gradually add Gaussian noise over T steps, producing increasingly noisy versions $x_1, \dots, x_T$. We use q to denote the distributions defined by this forward process. At each step:

$$x_t \sim \mathcal{N}(\sqrt{1 - \beta_t} x_{t-1}, \beta_t I) := q(x_t | x_{t-1}),$$

where $\beta_t \in (0, 1)$ is a small noise variance that follows a pre-defined schedule (typically increasing from $\beta_1 \approx 10^{-4}$ to $\beta_T \approx 0.02$). The $\sqrt{1 - \beta_t}$ factor shrinks the signal slightly at each step; we explain why below.

A key property is that we don't need to simulate all T steps sequentially. Let $\alpha_t = 1 - \beta_t$ (the fraction of signal preserved at step t) and $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$. Then we can sample x_t directly from x_0 in one shot (see Appendix A for derivation):

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \text{where } \epsilon \sim \mathcal{N}(0, I). \quad (1)$$

This says x_t is a weighted combination of the original image and random noise, or equivalently, $q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I)$.

The coefficients $\sqrt{\bar{\alpha}_t}$ and $\sqrt{1 - \bar{\alpha}_t}$ are chosen so that the total variance stays controlled: if x_0 has roughly unit variance, then $\text{Var}(x_t) \approx \bar{\alpha}_t \cdot \text{Var}(x_0) + (1 - \bar{\alpha}_t) \approx 1$ at every step, interpolating smoothly between the data variance and the noise variance. As t increases, $\bar{\alpha}_t \rightarrow 0$, so the signal x_0 fades and the noise dominates. At $t = 0$ we have the clean image; at $t = T$ we have $\bar{\alpha}_T \approx 0$ and $x_T \approx \epsilon$ — essentially pure Gaussian noise. The original image has been completely destroyed. This one-shot formula is also crucial for training, because it lets us instantly create a noisy sample x_t at any desired noise level by sampling a single ϵ .

3 Learning to Denoise

The goal of the reverse process is to start from noise $x_T \sim \mathcal{N}(0, I)$ and iteratively denoise to recover a clean image x_0 . The question is: what should the network learn to predict, and how do we turn a trained network into a sampler?

3.1 The Denoising Objective: Predicting x_0

The most intuitive formulation is to train a neural network $f_\theta(x_t, t)$ that takes a noisy image x_t and the noise level t , and directly predicts the original clean image x_0 .

The training procedure is simple:

1. Sample a training image x_0 from the dataset.
2. Pick a random timestep $t \sim \text{Uniform}\{1, \dots, T\}$.
3. Sample noise $\epsilon \sim \mathcal{N}(0, I)$ and create the noisy image: $x_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \epsilon$.
4. Feed x_t and t into the network, get prediction $\hat{x}_0 = f_\theta(x_t, t)$.
5. Minimize the loss: $\|x_0 - \hat{x}_0\|^2$.

The network never sees x_0 as input — only the noisy x_t . The clean image is the training *target*. Each time the same image appears in training, a fresh random t and fresh random ϵ produce a different x_t , so over training the network sees every image at every noise level.

Why does the network need t ? At different noise levels, the signal-to-noise ratio in x_t is very different. At $t \approx 0$, the image is nearly clean and the network just needs to remove a little noise. At $t \approx T$, the image is almost pure noise and the network must hallucinate plausible structure. If the network doesn't know t , it must apply a single compromise strategy across all noise levels. Telling it t lets it specialize: at high noise levels, focus on coarse structure; at low noise levels, focus on fine details. In practice, t is injected via a sinusoidal embedding (the same idea as positional encodings in transformers we will see in the next lecture), converted to a vector and added into each layer of the network.

Why not one giant step? If we train a network to map pure noise directly to a clean image (i.e., only use $t = T$), many different clean images all map to the same noise distribution — a picture of a “3” and a picture of a “7” both become indistinguishable Gaussian noise. The MSE loss pushes the network toward $\mathbb{E}[x_0|x_T]$, the average over all training images, producing the same blurry blob for every input. Breaking the process into many small steps avoids this: at intermediate noise levels, x_t still retains partial signal about which x_0 it came from, so the denoising problem is well-posed.

3.2 Network Architecture

What kind of network can take a noisy image x_t and a timestep t and output a clean-image estimate of the same spatial size? A common choice is the **U-Net** [RFB15] (Figure 2): an encoder that downsamples spatial dimensions while increasing channels, a decoder that upsamples back, and skip connections that let the decoder access fine details from the encoder. The timestep embedding is injected as a per-channel bias at each layer, so the network can adapt its behavior to the current noise level — making coarse corrections when the noise is heavy and fine corrections when the image is nearly clean.

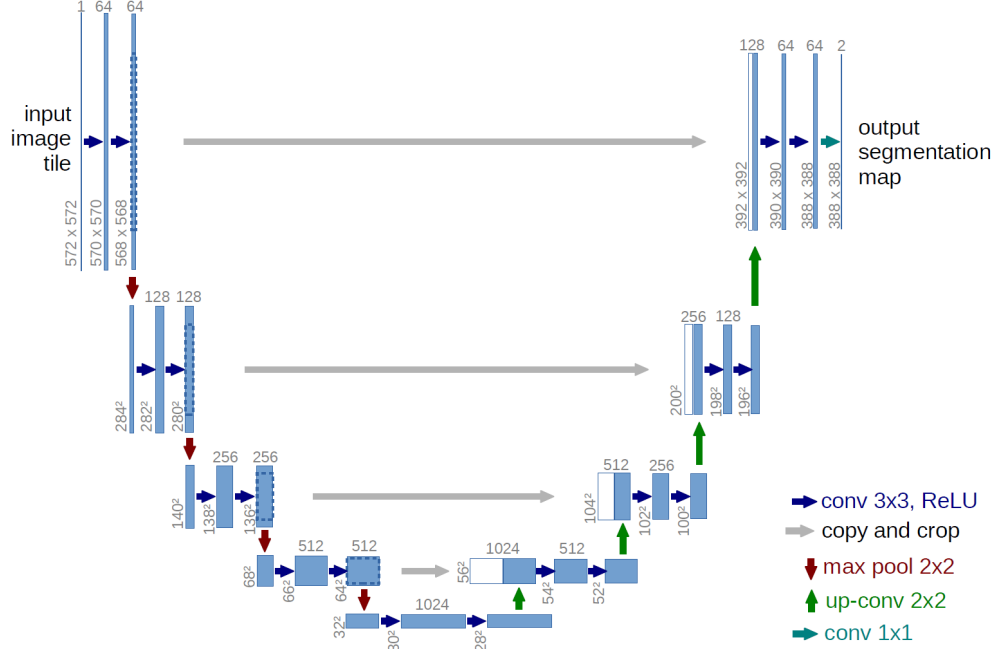


Figure 2: U-Net architecture commonly used for the denoising network. The network takes a noisy image x_t and timestep t and outputs a prediction of the same spatial dimensions. Source: [RFB15].

3.3 From Denoiser to Sampler

We have a network $f_\theta(x_t, t)$ that estimates x_0 from any noise level. How do we use it to generate images? We cannot just use $f_\theta(x_T, T)$ directly — at high noise the estimate is poor. Instead, we use an iterative procedure where each step refines the estimate:

1. **Start** with pure noise: $x_T \sim \mathcal{N}(0, I)$.
2. **Estimate** the clean image: $\hat{x}_0 = f_\theta(x_T, T)$. This estimate is rough, but it is a starting point.
3. **Back out the noise.** From the forward formula (Eq. 1), rearranging for ϵ :

$$\hat{\epsilon} = \frac{x_T - \sqrt{\bar{\alpha}_T} \hat{x}_0}{\sqrt{1 - \bar{\alpha}_T}}.$$

4. **Step to the next noise level.** We now have estimates of both the signal ($\hat{x}_0$) and the specific noise realization ($\hat{\epsilon}$) inside x_T . To construct a less noisy version, we plug the same $\hat{x}_0$ and $\hat{\epsilon}$ into the forward formula but with the $T-1$ coefficients, which put more weight on the signal:

$$x_{T-1} = \sqrt{\bar{\alpha}_{T-1}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{T-1}} \hat{\epsilon}.$$

5. **Re-estimate.** Feed x_{T-1} into the network at the new timestep: $\hat{x}_0 = f_\theta(x_{T-1}, T-1)$. This estimate is sharper, because the input has more signal.
6. **Repeat** until $t = 1$. At the final step, output $\hat{x}_0$.

At each step the network’s estimate of x_0 improves because the input carries more signal and less noise: early steps recover coarse structure (“this is a 7”) while later steps refine fine details (stroke thickness, slant). Since the update adds no fresh randomness, we can also skip timesteps to speed up generation — picking a subsequence of, say, 200 steps out of 1000 and applying the same formula. This idea is the basis of the DDIM sampler [SME20].

3.4 Equivalent Views: Noise Prediction and the ELBO

Everything above has used the *denoising view*: from x_t and t , the network predicts the clean image x_0 . The same learned reverse process can be parameterized in several equivalent ways. The original DDPM paper [HJA20] uses a different but equivalent parameterization: instead of predicting x_0 , the network predicts the *noise* ϵ that was added. The two views are interchangeable because the forward formula $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$ is a linear equation in two unknowns (x_0 and ϵ), so knowing one determines the other:

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \hat{\epsilon}}{\sqrt{\bar{\alpha}_t}}, \quad \hat{\epsilon} = \frac{x_t - \sqrt{\bar{\alpha}_t} \hat{x}_0}{\sqrt{1 - \bar{\alpha}_t}}.$$

The ϵ -prediction parameterization trains a network $\epsilon_\theta(x_t, t)$ to predict the noise, with loss:

$$L^{\text{simple}}(\theta) = E_{t, x_0, \epsilon} \left[\left\| \epsilon - \epsilon_\theta(\underbrace{\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon}_{x_t}, t) \right\|^2 \right], \quad (2)$$

where $t \sim U(1, T)$, $x_0 \sim q(x_0)$, and $\epsilon \sim \mathcal{N}(0, I)$. This is equivalent to the x_0 -prediction loss $\|x_0 - f_\theta(x_t, t)\|^2$ up to a reweighting across timesteps (the two losses assign different relative importance to different noise levels).

Why does ϵ -prediction work well in practice? The network’s target $\epsilon \sim \mathcal{N}(0, I)$ has the same scale at every timestep, which can make optimization more stable than predicting x_0 (whose scale varies with t). Both parameterizations are used in practice, and some recent methods (e.g., “v-prediction”) interpolate between the two. These are best understood as different parameterizations of the same reverse process, not different model classes.

Beyond this practical benefit, the ϵ -prediction loss has a principled justification: it can be derived from the Evidence Lower Bound (ELBO), connecting diffusion models to the same maximum likelihood framework used for VAEs. Recall that in a VAE we have a single latent variable z with a simple prior $p(z) = \mathcal{N}(0, I)$, a *learned* encoder $q_\phi(z|x)$ that maps data to latent space, and a learned decoder $p_\theta(x|z)$ that maps back. We optimize the ELBO:

$$\text{ELBO}_{\text{VAE}} = \underbrace{E_{q_\phi(z|x)}[\log p_\theta(x|z)]}_{\text{reconstruction}} - \underbrace{D_{KL}(q_\phi(z|x) \| p(z))}_{\text{prior matching}}.$$

A diffusion model has the same structure, but with two key differences:

- The “latent variables” are the entire chain of noisy images $x_1, x_2, \dots, x_T$, not a single z .
- The “encoder” is the *fixed* forward process $q(x_{1:T}|x_0)$ — just adding noise at each step. There is nothing to learn on the encoder side; all the learning is in the reverse process $p_\theta(x_{t-1}|x_t)$, which plays the role of the decoder.

The diffusion ELBO decomposes into T terms (see Appendix C), one for each denoising step. Each term asks the learned reverse step $p_\theta(x_{t-1}|x_t)$ to match the true reverse step $q(x_{t-1}|x_t, x_0)$ — analogous to the KL term in the VAE ELBO. Optimizing these terms is equivalent (up to reweighting) to the simple noise-prediction loss L^{simple} (Eq. 2).

The payoff of this connection is twofold. First, VAEs must jointly learn the encoder and decoder, and the quality of the latent space depends on how well $q_\phi(z|x)$ approximates the true posterior; diffusion models sidestep this entirely because the forward process is fixed, so all capacity goes into learning the reverse process. This is one reason diffusion models produce higher-quality samples than basic VAEs. Second, the ELBO derivation yields the DDPM sampling formula, which is the ϵ -prediction counterpart of our iterative loop from Section 3.3:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) + \sigma_t z, \quad z \sim \mathcal{N}(0, I),$$

where σ_t is a fixed variance schedule and $z = 0$ at the final step. This adds stochasticity for generating diverse samples, unlike the deterministic loop in Section 3.3.

4 Conditional Generation

So far the model generates random samples. To request a specific output — say “generate a 7” — we give the network a **condition** c (a class label, text embedding, etc.) as an additional input: $f_\theta(x_t, t, c)$. The mechanism is the same one used for the timestep: create a learned embedding that maps c to a vector and add it to the timestep embedding before injection into each layer. The loss is unchanged except that c is also an input:

$$\|x_0 - f_\theta(x_t, t, c)\|^2.$$

Classifier-free guidance. The dominant approach in practice is **classifier-free guidance** [HS22]. During training, the condition c is randomly dropped (replaced with a null token $\emptyset$) some fraction of the time, so the network learns both a conditional denoiser $f_\theta(x_t, t, c)$ and an unconditional denoiser $f_\theta(x_t, t, \emptyset)$. At sampling time, we combine the two x_0 -predictions:

$$\hat{x}_0 = f_\theta(x_t, t, \emptyset) + w \cdot [f_\theta(x_t, t, c) - f_\theta(x_t, t, \emptyset)],$$

where $w > 1$ amplifies the effect of the condition. Intuitively, the difference $f_\theta(x_t, t, c) - f_\theta(x_t, t, \emptyset)$ captures how the network’s estimate of x_0 shifts when it knows the desired condition. Scaling by $w > 1$ exaggerates this shift, producing samples that more strongly reflect c at the cost of some diversity. This guided $\hat{x}_0$ is then used in the same iterative sampling loop from Section 3.3. This is what enables models like DALL-E and Stable Diffusion to follow text prompts.

5 Extensions

Score-based view. There is an alternate interpretation of diffusion models as performing *score matching*: learning the gradient of the log probability density $\nabla_x \log p(x_t)$ at each noise level. This is another lens on the same reverse process, not a fundamentally different model family. The score

function points toward higher-density regions and can be used to iteratively refine samples via Langevin dynamics. See this blog by Yang Song and *The Principles of Diffusion Models* [LSK⁺25] for a unified treatment.

Flow-based view. At a higher level, one can also view diffusion as transporting a simple prior to the data distribution along a continuous path under a learned time-dependent velocity field. This viewpoint helps unify diffusion, score-based models, and more recent flow-matching methods. We will not develop that perspective here, but it is a useful modern way to see these methods as part of one larger family.

Slow sampling. Generating an image requires running the network T times (e.g., $T = 1000$). This is addressed by DDIM [SME20] (which allows skipping steps), consistency models [SDCS23] (learning a one-step or few-step mapping), and faster ODE solvers.

Latent diffusion. Running diffusion at full image resolution is expensive. **Latent Diffusion Models** (LDMs) [RBL⁺22] address this by first training a VAE to compress images into a lower-dimensional latent space, then running the diffusion process in that latent space. This is the basis for Stable Diffusion.

References

- [HJA20] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.
- [HS22] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022.
- [LSK⁺25] Chieh-Hsin Lai, Yang Song, Dongjun Kim, Yuki Mitsufuji, and Stefano Ermon. The principles of diffusion models. *arXiv preprint arXiv:2510.21890*, 2025.
- [RBL⁺22] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022.
- [RFB15] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical image computing and computer-assisted intervention—MICCAI 2015: 18th international conference, Munich, Germany, October 5–9, 2015, proceedings, part III 18*, pages 234–241. Springer, 2015.
- [SDCS23] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. In *Proceedings of the 40th International Conference on Machine Learning*, pages 32211–32252, 2023.
- [SME20] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502*, 2020.

(Optional reading: The following sections provide mathematical derivations for results presented in the main text.)

A Derivation of $q(x_t|x_0)$

Let $\alpha_t = 1 - \beta_t$. The single step definition is $x_t = \sqrt{\alpha_t}x_{t-1} + \sqrt{1 - \alpha_t}\epsilon_{t-1}$, where $\epsilon_{t-1} \sim \mathcal{N}(0, I)$. Applying this recursively:

$$\begin{aligned} x_t &= \sqrt{\alpha_t}x_{t-1} + \sqrt{1 - \alpha_t}\epsilon_{t-1} \\ &= \sqrt{\alpha_t}(\sqrt{\alpha_{t-1}}x_{t-2} + \sqrt{1 - \alpha_{t-1}}\epsilon_{t-2}) + \sqrt{1 - \alpha_t}\epsilon_{t-1} \\ &= \sqrt{\alpha_t\alpha_{t-1}}x_{t-2} + \sqrt{\alpha_t(1 - \alpha_{t-1})}\epsilon_{t-2} + \sqrt{1 - \alpha_t}\epsilon_{t-1} \end{aligned}$$

Recall that the sum of two independent Gaussian random variables $\mathbf{z}_1 \sim \mathcal{N}(0, \sigma_1^2 I)$ and $\mathbf{z}_2 \sim \mathcal{N}(0, \sigma_2^2 I)$ is also Gaussian: $\mathbf{z}_1 + \mathbf{z}_2 \sim \mathcal{N}(0, (\sigma_1^2 + \sigma_2^2)I)$. Let $\mathbf{z}_1 = \sqrt{\alpha_t(1 - \alpha_{t-1})}\epsilon_{t-2}$ and $\mathbf{z}_2 = \sqrt{1 - \alpha_t}\epsilon_{t-1}$. Since ϵ_{t-1} and ϵ_{t-2} are independent samples from $\mathcal{N}(0, I)$, the sum $\mathbf{z}_1 + \mathbf{z}_2$ is distributed as $\mathcal{N}(0, [\alpha_t(1 - \alpha_{t-1}) + (1 - \alpha_t)]I) = \mathcal{N}(0, (1 - \alpha_t\alpha_{t-1})I)$. This means we can rewrite the sum of noise terms as a single noise term $\sqrt{1 - \alpha_t\alpha_{t-1}}\bar{\epsilon}_{t-2}$, where $\bar{\epsilon}_{t-2} \sim \mathcal{N}(0, I)$. So,

$$x_t = \sqrt{\alpha_t\alpha_{t-1}}x_{t-2} + \sqrt{1 - \alpha_t\alpha_{t-1}}\bar{\epsilon}_{t-2}.$$

Letting $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$. Repeating this process recursively all the way back to x_0 , we get:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, \quad \text{where } \epsilon \sim \mathcal{N}(0, I).$$

This is equivalent to saying:

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I).$$

B Derivation of $q(x_{t-1}|x_t, x_0)$

Using Bayes' theorem:

$$q(x_{t-1}|x_t, x_0) = q(x_t|x_{t-1}, x_0) \frac{q(x_{t-1}|x_0)}{q(x_t|x_0)}.$$

Since x_t only depends on x_{t-1} (Markov property of the forward process), $q(x_t|x_{t-1}, x_0) = q(x_t|x_{t-1})$. Plugging in the Gaussian forms for $q(x_t|x_{t-1})$, $q(x_{t-1}|x_0)$, and $q(x_t|x_0)$ derived from the forward process:

$$\begin{aligned} q(x_t|x_{t-1}) &= \mathcal{N}(x_t; \sqrt{\alpha_t}x_{t-1}, \beta_t I) \\ q(x_{t-1}|x_0) &= \mathcal{N}(x_{t-1}; \sqrt{\bar{\alpha}_{t-1}}x_0, (1 - \bar{\alpha}_{t-1})I) \\ q(x_t|x_0) &= \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I) \end{aligned}$$

Substituting these into the Bayes' formula and simplifying the resulting product of Gaussians in the exponent (a lengthy but standard algebraic manipulation involving completing the square) yields a Gaussian distribution for $q(x_{t-1}|x_t, x_0)$ with mean and variance:

$$\tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t$$

$$\tilde{\mu}(x_t, x_0) = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t} x_0.$$

Substituting $x_0 = \frac{1}{\sqrt{\bar{\alpha}_t}}(x_t - \sqrt{1 - \bar{\alpha}_t}\epsilon)$ (rearranging the forward process formula) into the mean expression gives an alternative form:

$$\tilde{\mu}(x_t, x_0) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_t \right).$$

This shows the mean depends explicitly on the noise ϵ_t used to generate x_t from x_0 .

C Maximum Likelihood Perspective

Recall from last lecture that for VAEs, the Evidence Lower Bound (ELBO) on the log-likelihood $\log p_\theta(x)$ is given by:

$$L_{\text{VAE}} = E_{q_\phi(z|x)} [\log p_\theta(x|z)] - D_{KL}(q_\phi(z|x) || p(z)).$$

For diffusion models, we also optimize an ELBO, but the derivation accounts for the Markov chain structure. We start with the definition of the ELBO for the full sequence $x_{0:T}$:

$$L_{\text{VLB}} = E_{q(x_{1:T}|x_0)} \left[\log \frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)} \right].$$

Expanding the joint distributions $p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^T p_\theta(x_{t-1}|x_t)$ and $q(x_{1:T}|x_0) = \prod_{t=1}^T q(x_t|x_{t-1})$:

$$\begin{aligned} L_{\text{VLB}} &= -E_q \left[\log \frac{\prod_{t=1}^T q(x_t|x_{t-1})}{p_\theta(x_T) \prod_{t=1}^T p_\theta(x_{t-1}|x_t)} \right] \\ &= E_q \left[\log \frac{p_\theta(x_T) p_\theta(x_0|x_1)}{q(x_1|x_0)} \right] - E_q \left[\log \prod_{t=2}^T \frac{q(x_t|x_{t-1})}{p_\theta(x_{t-1}|x_t)} \right] \\ &= E_q \left[\log \frac{p_\theta(x_T) p_\theta(x_0|x_1)}{q(x_1|x_0)} \right] - E_q \left[\log \prod_{t=2}^T \frac{q(x_t|x_{t-1}, x_0)}{p_\theta(x_{t-1}|x_t)} \right] \\ &= E_q \left[\log \frac{p_\theta(x_T) p_\theta(x_0|x_1)}{q(x_1|x_0)} \right] - E_q \left[\log \prod_{t=2}^T \frac{q(x_{t-1}|x_t, x_0) q(x_t|x_0)}{q(x_{t-1}|x_0) p_\theta(x_{t-1}|x_t)} \right] \\ &= E_q \left[\log \frac{p_\theta(x_T) p_\theta(x_0|x_1)}{q(x_1|x_0)} \right] - E_q \left[\log \frac{q(x_T|x_0)}{q(x_1|x_0)} \right] - \sum_{t=2}^T E_q \left[\log \frac{q(x_{t-1}|x_t, x_0)}{p_\theta(x_{t-1}|x_t)} \right] \\ &= E_q [\log p_\theta(x_0|x_1)] - E_q \left[\log \frac{q(x_T|x_0)}{p_\theta(x_T)} \right] - \sum_{t=2}^T E_q \left[\log \frac{q(x_{t-1}|x_t, x_0)}{p_\theta(x_{t-1}|x_t)} \right] \\ &= \underbrace{E_q [\log p_\theta(x_0|x_1)]}_{L_0} - \sum_{t=2}^T \underbrace{D_{KL}(q(x_{t-1}|x_t, x_0), p_\theta(x_{t-1}|x_t))}_{L_{t-1}} - \underbrace{D_{KL}(q(x_T|x_0), p_\theta(x_T))}_{L_T} \end{aligned}$$

Let us interpret the terms above:

- L_0 is the reconstruction term $E_{q(x_1|x_0)} [\log p_\theta(x_0|x_1)]$.
- L_T measures how close the distribution of the final noisy input is to the standard Gaussian prior. This has no trainable parameters.
- L_{t-1} for $t = 2, \dots, T$ are the KL-divergence terms $D_{KL}(q(x_{t-1}|x_t, x_0) || p_\theta(x_{t-1}|x_t))$ that match the learned reverse step to the true one.

The key terms for training are L_{t-1} . Let's focus on one such term, dropping the expectation E_q for now. This is the KL divergence between two Gaussians: $q(x_{t-1}|x_t, x_0) = \mathcal{N}(\tilde{\mu}(x_t, x_0), \tilde{\beta}_t I)$ and $p_\theta(x_{t-1}|x_t) = \mathcal{N}(\mu_\theta(x_t, t), \sigma_t^2 I)$. As discussed, we typically fix the variance of the model σ_t^2 to match the true posterior variance, i.e., $\sigma_t^2 = \tilde{\beta}_t$. In this case, the KL divergence simplifies significantly:

$$L_{t-1} = D_{KL} \left(\mathcal{N}(\tilde{\mu}, \tilde{\beta}_t I) || \mathcal{N}(\mu_\theta, \tilde{\beta}_t I) \right) = E_{q(x_t|x_0)} \left[\frac{1}{2\tilde{\beta}_t} \|\tilde{\mu}(x_t, x_0) - \mu_\theta(x_t, t)\|^2 \right] + \text{constants}$$

(Note: If σ_t^2 is fixed differently, e.g., $\sigma_t^2 = \beta_t$, there are additional constant terms, but the dependence on the means remains).

Now, substitute the expressions for the means in terms of ϵ_t and ϵ_θ :

$$\begin{aligned} \tilde{\mu}(x_t, x_0) &= \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_t \right) \\ \mu_\theta(x_t, t) &= \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) \end{aligned}$$

Plugging these into the KL expression gives:

$$\begin{aligned} L_{t-1} &= E_{q(x_t|x_0)} \left[\frac{1}{2\tilde{\beta}_t} \left\| \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_t \right) - \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) \right\|^2 \right] + \dots \\ &= E_{q(x_t|x_0), \epsilon_t \sim \mathcal{N}(0, I)} \left[\frac{1}{2\tilde{\beta}_t} \left\| \frac{1}{\sqrt{\alpha_t}} \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} (\epsilon_\theta(x_t, t) - \epsilon_t) \right\|^2 \right] + \dots \\ &= E_{x_0, \epsilon_t, t} \left[\frac{\beta_t^2}{2\tilde{\beta}_t \alpha_t (1 - \bar{\alpha}_t)} \|\epsilon_t - \epsilon_\theta(x_t, t)\|^2 \right] + \dots \end{aligned}$$

The term L_0 can also be related to $\|\epsilon_1 - \epsilon_\theta(x_1, 1)\|^2$. Summing all terms $L_0 + \dots + L_{T-1}$ results in a weighted sum of squared errors between the true and predicted noise at different timesteps. Ho et al. [HJA20] found that using a simplified objective, which ignores the weighting factors $\frac{\beta_t^2}{2\tilde{\beta}_t \alpha_t (1 - \bar{\alpha}_t)}$, works well in practice and is easier to implement. This leads to the simple objective:

$$L^{\text{simple}} = E_{t \sim U(1, T), x_0 \sim q(x_0), \epsilon_t \sim \mathcal{N}(0, I)} [\|\epsilon_t - \epsilon_\theta(\sqrt{\alpha_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon_t, t)\|^2]$$